



UNIVERSIDADE ESTADUAL DE MARINGÁ
DEPARTAMENTO DE INFORMÁTICA
BACHARELADO EM ENGENHARIA DE SOFTWARE



ISABELLA BARELA VENÂNCIO | RA144023
PEDRO AUGUSTO DOS SANTOS WATERMANN | RA145121
VINÍCIUS GARCIA DEL CIELO | RA143653
RAUL GONÇALVES FERREIRA | RA143927
GUSTAVO BRAGADA CAVALCANTI | RA145116

DOCUMENTAÇÃO TÉCNICA DE SISTEMA ROLE-PLAYING GAME (RPG):

Interface Gráfica e Persistência

Maringá
2026

SUMÁRIO

1. Introdução e Evolução do Projeto.....	3
1.1. Breve descrição do contexto escolhido.....	3
1.2. Modificações e Adaptações da Modelagem.....	3
2. Interface Gráfica (View).....	4
2.1. Tecnologia Utilizada.....	4
3. Persistência de Dados.....	5
3.1. Estratégia e Tecnologias.....	5
3.2. Mapeamento Objeto-Relacional (ORM), JPA, Hibernate e SQLite.....	5
4. Implementação e Execução.....	6
4.1. Organização do Código.....	6
4.2. Demonstração da interface gráfica e persistência dos dados.....	9

1. Introdução e Evolução do Projeto

1.1. Breve descrição do contexto escolhido

O projeto representa o cenário de gerenciamento de campanhas de RPG de mesa, inspirado no sistema Dungeons & Dragons. Nesse contexto, jogadores e mestres precisam lidar com uma grande quantidade de informações, como fichas de personagens, missões e registros da campanha.

O problema central está na falta de organização e centralização desses dados, que normalmente são controlados de forma manual ou em ferramentas separadas, como anotações físicas e planilhas. Isso pode resultar em perda de informações, inconsistências e dificuldade na continuidade das campanhas.

Assim, o sistema proposto busca representar e solucionar esse cenário por meio de uma plataforma digital integrada que centraliza e organiza todas as informações, tornando o gerenciamento mais eficiente e melhorando a experiência dos participantes.

1.2. Modificações e Adaptações da Modelagem

Durante a transição do modelo conceitual para a implementação prática na arquitetura do software, foram necessárias diversas modificações e adaptações estruturais. As principais adaptações aplicadas foram:

1. Adoção do Padrão Arquitetural MVC (Model-View-Controller): O projeto foi inteiramente reestruturado seguindo o padrão MVC de forma modularizada por contexto de negócio (ex: pacotes separados para Campanha, Personagem, Arma, Usuario, etc.). Cada módulo possui suas próprias camadas de model (entidades e dados), view (telas Swing) e controller (regras de negócio e mediação). Isso garante alta coesão, baixo acoplamento e facilita a manutenção do sistema à medida que novas regras de D&D forem adicionadas.
2. Mapeamento Objeto-Relacional (ORM) com JPA/Hibernate: Para garantir que o progresso das campanhas não seja perdido, integrou-se a especificação Jakarta Persistence (JPA) com o provedor Hibernate. No arquivo persistence.xml, foi configurada a propriedade hibernate.hbm2ddl.auto como update. Isso adaptou o modelo de dados para que o próprio Java crie e sincronize as tabelas no banco de dados PostgreSQL automaticamente a partir das anotações contidas nas classes

@Entity, traduzindo conceitos de Orientação a Objetos diretamente para o modelo relacional.

3. Modularização da Ficha de Personagem através de Composição: O modelo foi adaptado aplicando o princípio de composição de objetos. Informações complexas foram decompostas em entidades próprias, tais como AtributoModel (para os atributos de força, destreza, etc.), ClasseModel, RacaModel, ArmaModel e MagiaModel. Elas se relacionam com o personagem por meio de mapeamentos do JPA (como @OneToOne e @ManyToMany), permitindo que múltiplos personagens compartilhem dados padronizados de raças e classes sem duplicar registros desnecessários no banco.
4. Tratamento de Herança e Papéis de Usuários: O modelo de usuários foi adaptado para herança de classes. A partir de uma classe base UsuarioModel, derivam-se especializações como MestreModel e JogadorModel. Isso permitiu amarrar regras de multiplicidade no banco de dados (por exemplo: uma CampanhaModel possui um relacionamento @ManyToOne com um Mestre, mas possui um relacionamento @ManyToMany com uma lista de Jogadores), refletindo com exatidão a dinâmica real de uma mesa de RPG.
5. Evolução da Interface de Usuário (CLI para GUI): Para cumprir a proposta de ser uma plataforma integrada e intuitiva, toda a interação foi portada para uma Interface Gráfica de Usuário (GUI) usando Java Swing. Foram criadas views dedicadas para listagem (CampanhaListView), criação (CampanhaCreateView) e seleção (CampanhaSelectView), além de componentes visuais personalizados (como classes para bordas arredondadas), garantindo que usuários que não possuem afinidade com tecnologia consigam gerenciar suas mesas visualmente de forma simples.

2. Interface Gráfica (View)

2.1. Tecnologia Utilizada

Para o desenvolvimento da interface gráfica do sistema de gerenciamento de campanhas de RPG, optamos pela utilização da biblioteca Java Swing. A escolha baseou-se nos seguintes critérios: Natividade e Compatibilidade, sendo uma API padrão integrada ao Java, o Swing dispensa a necessidade de configurações complexas de dependências externas ou módulos adicionais no ambiente de execução, garantindo que o projeto compile e execute de forma direta.

Adequação ao Escopo, o Swing oferece um conjunto de componentes robusto e tradicional (como JFrame, JPanel, JButton, JTextField e JTable), sendo suficiente para atender aos objetivos e o casamento com a Arquitetura MVC, a tecnologia facilita a separação estrita da camada de View (Visão), permitindo mapear os eventos dos componentes visuais (via ActionListener) diretamente para os métodos da camada de Controller (Controlador), sem misturar as regras de negócio ou lógica de persistência dentro das telas.

3. Persistência de Dados

3.1. Estratégia e Tecnologias

Para a persistência de dados do sistema, foi adotada uma estratégia baseada em banco de dados relacional remoto, utilizando o Supabase como plataforma de hospedagem, que provê uma instância gerenciada do PostgreSQL. Assim, a escolha por um banco de dados remoto em vez de uma solução local justifica-se pelo contexto da aplicação: sistemas de gerenciamento de campanhas de RPG envolvem múltiplos jogadores que precisam acessar e compartilhar dados em tempo real, tornando uma solução centralizada e acessível pela rede a mais adequada ao problema.

A integração técnica entre a aplicação Java e o servidor de banco de dados é realizada utilizando o driver oficial do PostgreSQL via JDBC (Java Database Connectivity). Para garantir a segurança das credenciais de acesso à infraestrutura na nuvem, o projeto integra a biblioteca dotenv-java, permitindo que o host, a porta e as senhas sejam injetados dinamicamente a partir de variáveis de ambiente (.env). Em termos de execução, em vez de manipular instruções SQL brutas e gerenciar objetos de conexão de forma manual na camada de negócios, toda a comunicação de baixo nível é delegada a um ecossistema automatizado de persistência, o qual se encarrega de parametrizar as requisições de forma transparente e blindar a aplicação contra vulnerabilidades de injeção de SQL.

3.2. Mapeamento Objeto-Relacional (ORM), JPA, Hibernate e SQLite

O sistema emprega o conceito de Mapeamento Objeto-Relacional (ORM), mapeando as tabelas da base de dados diretamente em classes Java, tais como CampanhaModel, ArmaModel e AtributoModel. Essa arquitetura é padronizada pela especificação JPA 3.1.0 através do uso de anotações estruturais (@Entity, @Table, @Id, @ManyToOne), tendo as suas operações e ciclo de vida geridos pelo EntityManager por intermédio da classe utilitária JpaUtil.

O Hibernate atua como o provedor prático da JPA, traduzindo as instruções orientadas a objetos e as consultas escritas em JPQL para o dialeto nativo do PostgreSQL. O framework também automatiza a criação e sincronização do esquema em tempo de execução através da propriedade

hibernate.hbm2ddl.auto=update declarada no arquivo persistence.xml. Por fim, o uso do SQLite foi descartado para o escopo principal, uma vez que, por ser um banco de dados local baseado num único arquivo, carece da arquitetura cliente-servidor exigida pelo ecossistema multijogador em rede. Contudo, devido ao nível de abstração fornecido pela JPA e pelo Hibernate, uma futura transição para o SQLite visando um modo offline exigiria apenas a alteração do driver e do dialeto no persistence.xml, preservando todo o código de negócio e controladores já desenvolvidos.

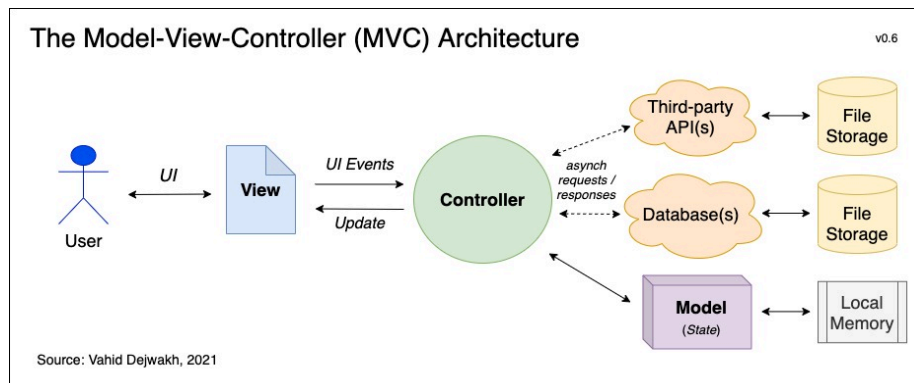
4. Implementação e Execução

4.1. Organização do Código

- Arquitetura de Software: Model-View-Controller (MVC)

O padrão MVC foi adotado para garantir a separação de preocupações (separation of concerns), permitindo que a lógica de negócios, a interface do usuário e o controle de dados operem de forma independente.

- Model (Modelo): Gerencia o comportamento, as regras de negócio e os dados do domínio. É representado por classes de entidade (ex: CampanhaModel) mapeadas via anotações JPA para o banco de dados.
- View (Visão): Camada de interface gráfica desenvolvida em Java Swing (ex: CampanhaCreateView), responsável por renderizar os dados e capturar os eventos e comandos do usuário.
- Controller (Controlador): Atua como intermediário direto (ex: CampanhaCreateController), interceptando as ações da View, coordenando as regras de negócio através do Model e atualizando a interface com as respostas do sistema.



- Gerenciamento de Dependências e Build: Maven

A automação do ciclo de vida da aplicação e o gerenciamento de bibliotecas externas são realizados através do Apache Maven, estruturado no arquivo pom.xml. O Maven elimina a necessidade de incluir arquivos JAR manualmente no projeto, encarregando-se de baixar, estender e organizar as dependências essenciais do ecossistema, como o Hibernate Core (provedor ORM), a especificação Jakarta Persistence (JPA 3.1.0), o driver JDBC do PostgreSQL e a biblioteca de gerenciamento de ambiente.

- Interface Gráfica de Usuário (GUI): Java Swing

A experiência visual e as telas do sistema são construídas com a API Java Swing, fornecendo uma aplicação desktop nativa e intuitiva para mestres e jogadores. O fluxo de navegação é segmentado por visões de propósito único, englobando componentes para listagem, criação e seleção de registros (como CampanhaListView e CampanhaCreateView). Para garantir uma identidade visual agradável, foram implementadas customizações estéticas diretamente no código, como classes utilitárias para renderização de bordas arredondadas nos elementos da interface.

- Configuração e Segurança do Ambiente: Dotenv

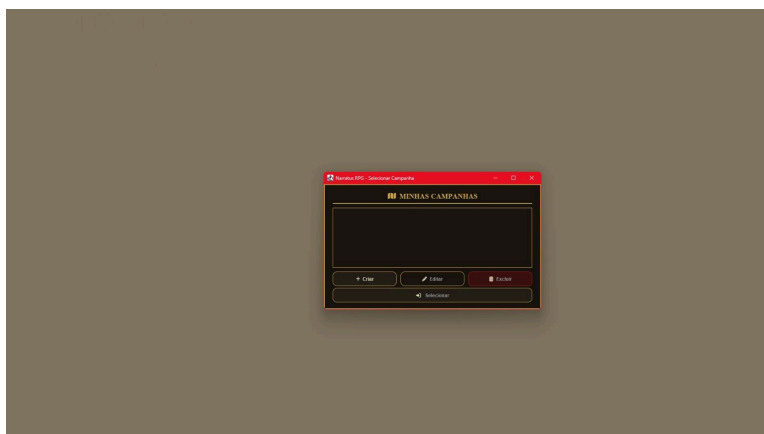
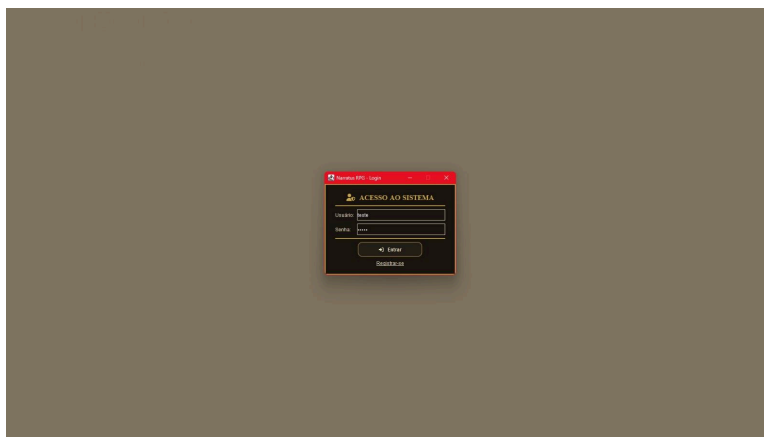
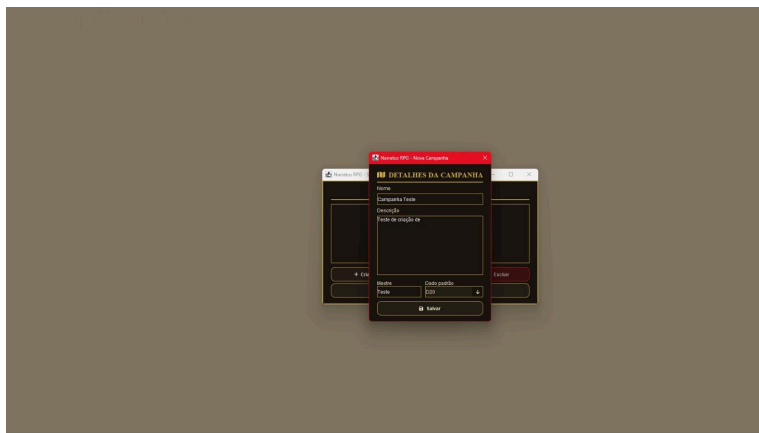
A execução segura da aplicação depende do isolamento de dados sensíveis da infraestrutura de banco de dados. Para isso, o projeto integra a biblioteca dotenv-java, que permite carregar variáveis de ambiente a partir de um arquivo local .env. Parâmetros críticos como o host do Supabase, porta, usuário e senha de produção não ficam expostos no código-fonte, sendo substituídos por um arquivo modelo .env.example no repositório, garantindo conformidade com as boas práticas de segurança em desenvolvimento de software.

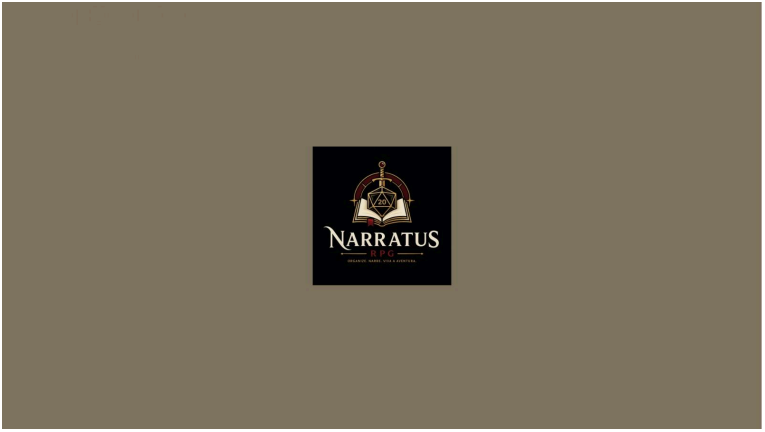
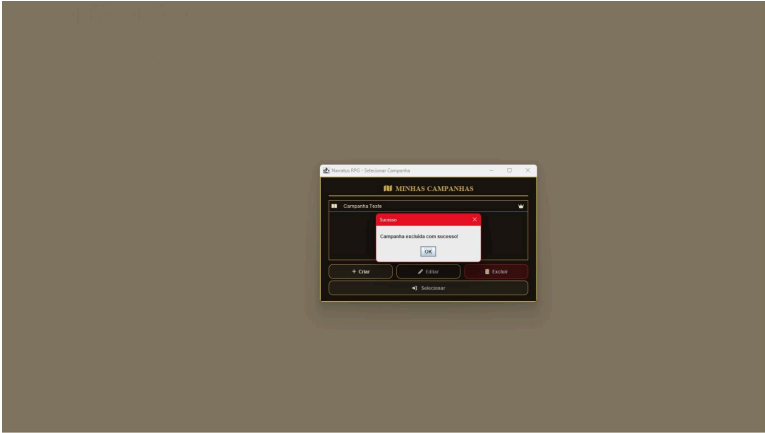
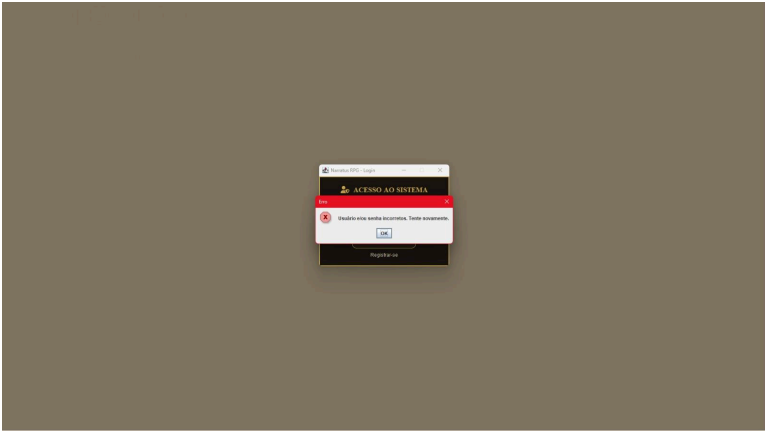
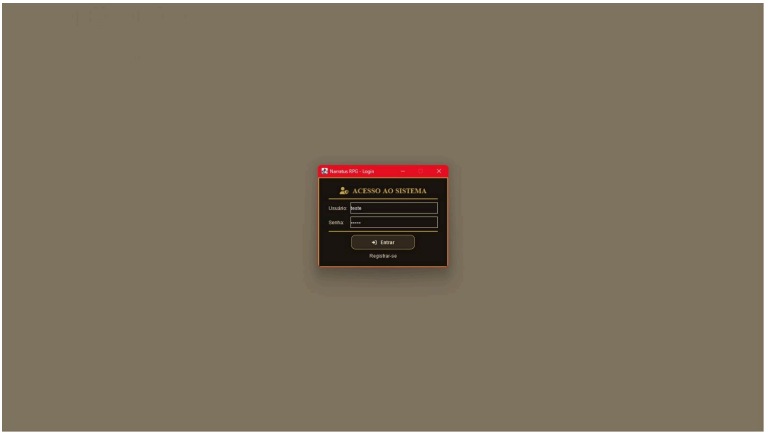
- Gestão de Código e Versionamento: GitHub

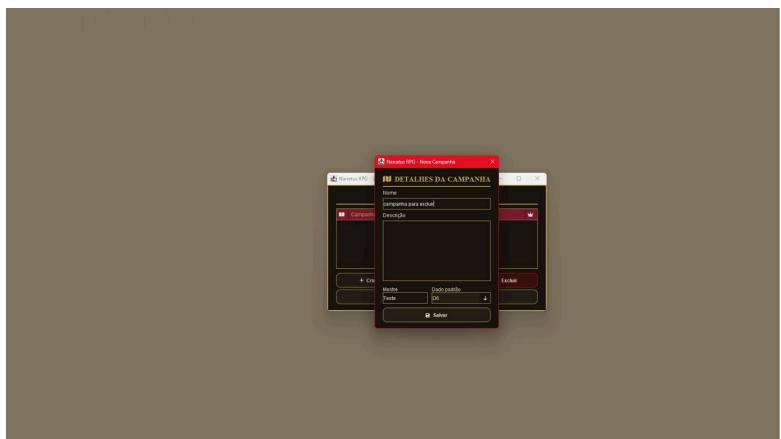
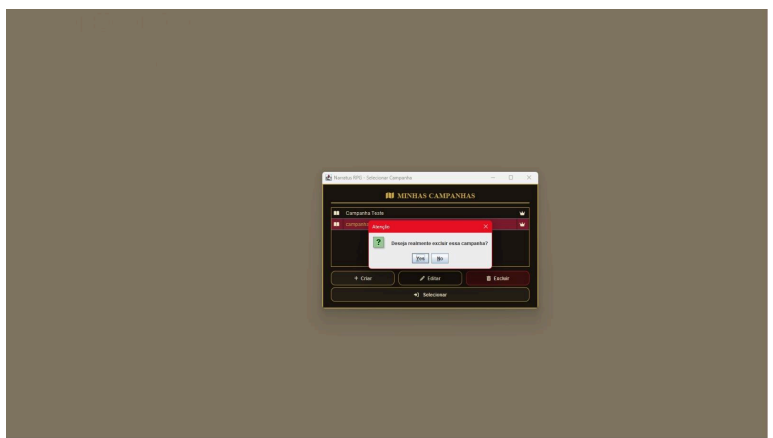
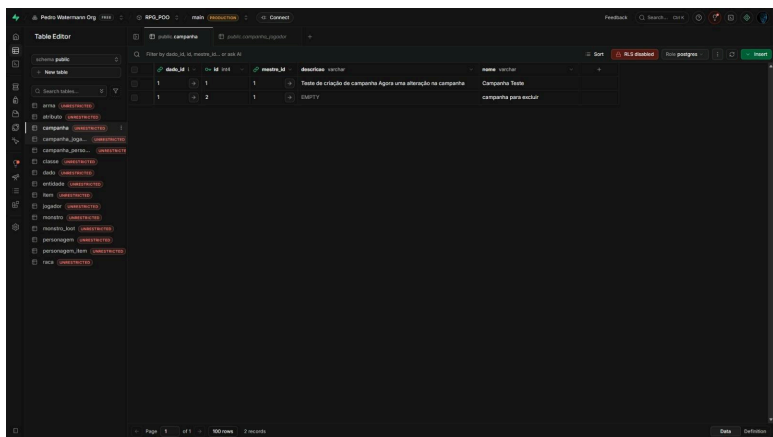
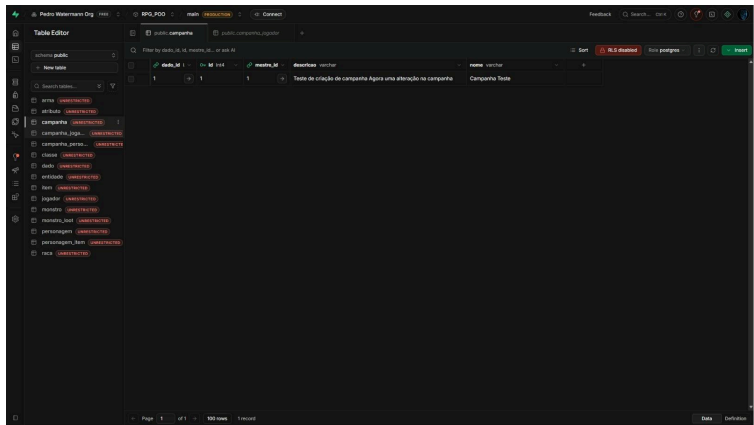
A execução do projeto e a colaboração foram gerenciadas através do GitHub, utilizando o sistema de controle de versão Git.

- Versionamento: Controle do histórico de alterações.
- Trabalho em Equipe: Uso de Branches para o desenvolvimento de novas funcionalidades sem comprometer o código principal (Master).
- Code Review: Utilização de Pull Requests para revisão de código, garantindo a qualidade e a padronização antes da integração final.
- Documentação: O repositório centraliza não apenas o código-fonte, mas também o arquivo README.md com instruções de instalação e execução.

4.2. Demonstração da interface gráfica e persistência dos dados







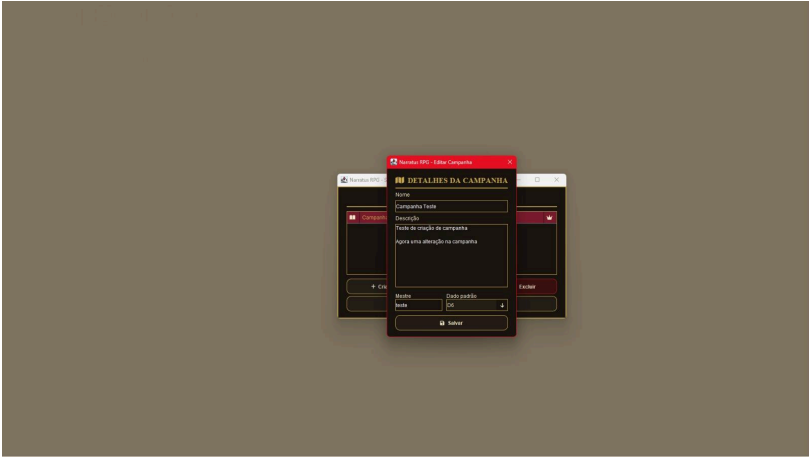
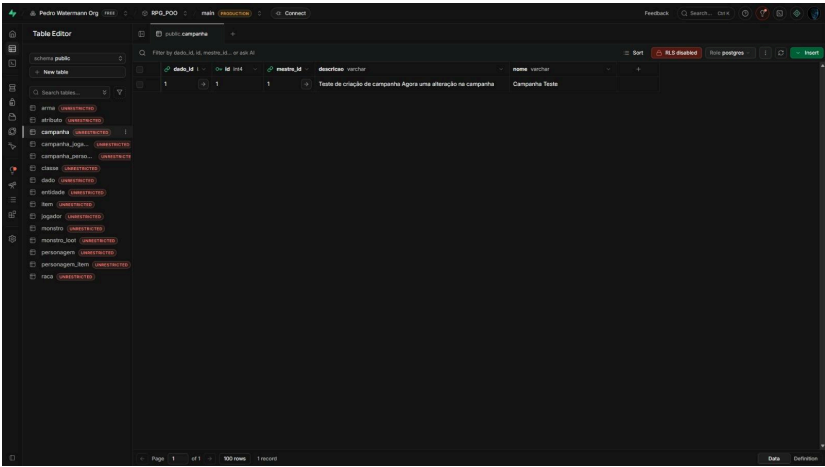
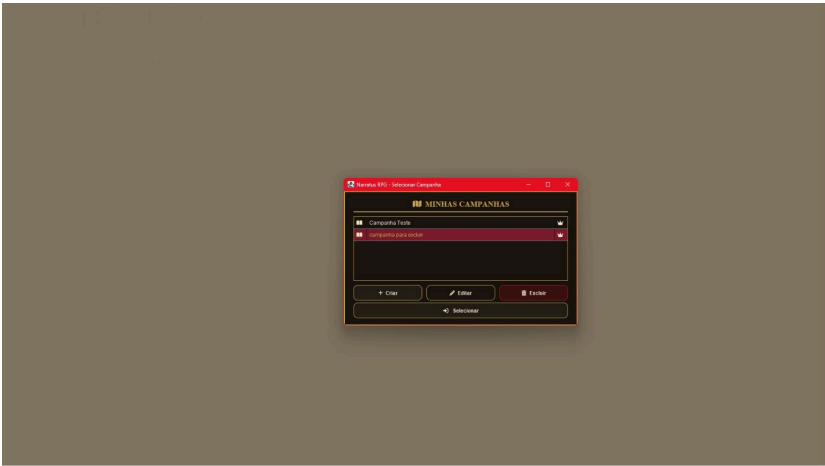


Table Editor

public campanha

Filter by attribute, id, name, or value

Sort **RL disabled** **Rows progress** **Insert**

id	atributo_id	valor	descricao	nome
1	1	1	Teste de criação de	Campanha Teste

Search tables

- arvore (000176170)
- atributo (000176170)
- campanha (000176170)
- campanha_jogo (000176170)
- campanha_parte (000176170)
- classe (000176170)
- dado (000176170)
- entidade (000176170)
- item (000176170)
- jogador (000176170)
- monstro (000176170)
- monstro_boss (000176170)
- personagem (000176170)
- personagem_boss (000176170)
- regra (000176170)

Page 1 of 1 100 rows 1 record Data Definition

Table Editor

public classe

Filter by attribute, id, name, or value

Sort **RL disabled** **Rows progress** **Insert**

id	atributo_id	valor	descricao
1	1	6	
2	2	20	
3	3	12	
4	4	100	

Search tables

- arvore (000176170)
- atributo (000176170)
- campanha (000176170)
- campanha_jogo (000176170)
- campanha_parte (000176170)
- classe (000176170)
- dado (000176170)
- entidade (000176170)
- item (000176170)
- jogador (000176170)
- monstro (000176170)
- monstro_boss (000176170)
- personagem (000176170)
- personagem_boss (000176170)
- regra (000176170)

Page 1 of 1 100 rows 4 records Data Definition

Table Editor

public jogador

Filter by id, name, attribute or value

Sort **RL disabled** **Rows progress** **Insert**

id	nome	posicao
1	teste	teste

Search tables

- arvore (000176170)
- atributo (000176170)
- campanha (000176170)
- campanha_jogo (000176170)
- campanha_parte (000176170)
- classe (000176170)
- dado (000176170)
- entidade (000176170)
- item (000176170)
- jogador (000176170)
- monstro (000176170)
- monstro_boss (000176170)
- personagem (000176170)
- personagem_boss (000176170)
- regra (000176170)

Page 1 of 1 100 rows 1 record Data Definition